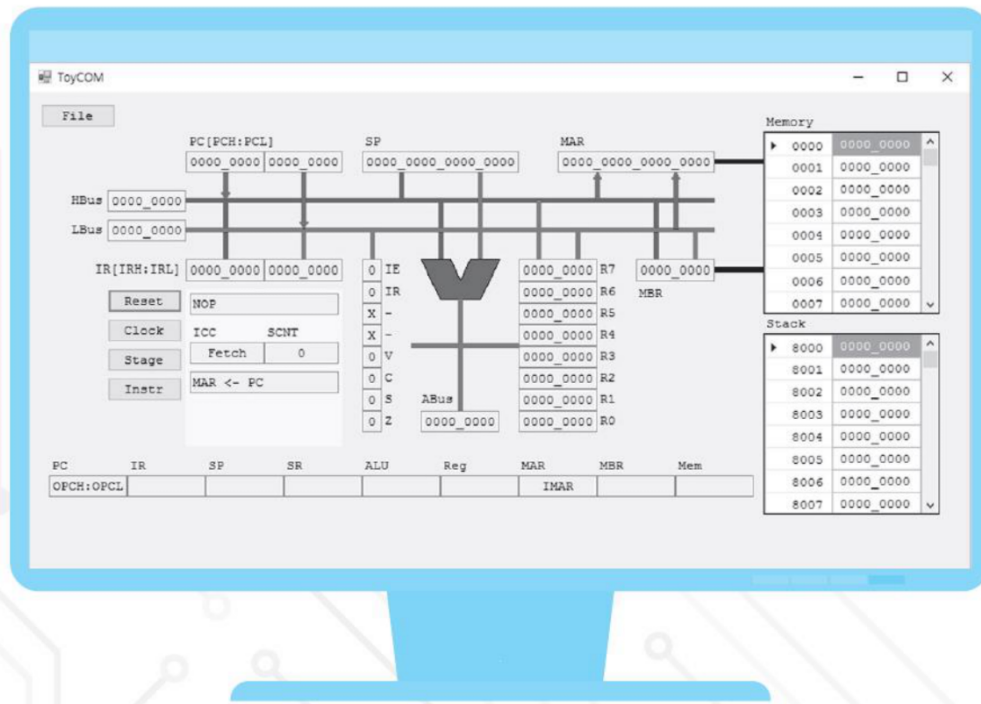


한눈에 보이는

개정 3 판

컴퓨터 구조



05. 연산기

5.1 연산기 개요

5.2 정수 표현

5.3 논리 연산

5.4 시프트 연산

5.5 산술 연산

5.6 실수 표현

5.7 실수 연산

5.8 요약

5.1 연산기 개요

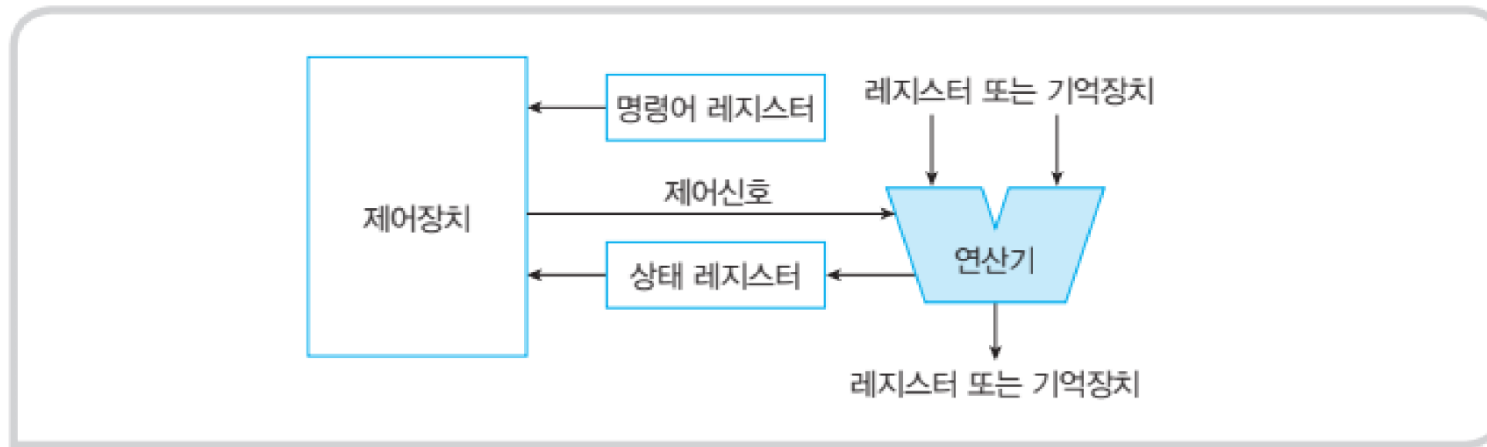


그림 5-1 연산기 구조

연산의 종류

- 단항 연산자 (unary operator)
 - -(음수 만들기), 1의 보수(NOT), 왼쪽/오른쪽 시프트, 증가, 감소
- 이항 연산자 (binary operator)
 - 사칙 연산(+, -, x, /), 논리 연산(AND, OR, XOR), 비교(compare, test)

컴퓨터의 수

표 5-1 수의 표현과 연산 방법

수의 형식	표현	연산 방법
부호 없는 수	2.1 수와 코드	논리 연산, 논리시프트 연산, 산술 연산 <i>unsigned</i>
부호 있는 수(정수)	5.2 정수 표현	산술시프트 연산, 산술 연산 <i>signed</i>
실수	5.6 실수 표현	산술 연산 <i>float/double</i>

5.2 정수 표현

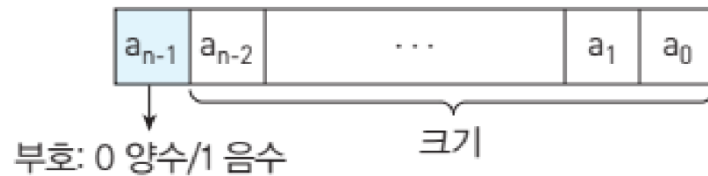
소수점이 없는 수

- 부호 없는 수(unsigned number): 2장
- 정수(signed number): 5.2절

정수 표현 방법

- 5.2.1 부호화 크기 (signed magnitude) → 양수: MSB 0, 음수: MSB 1
- 5.2.2 보수 (1의 보수 의미)
- 5.2.3 2의 보수 - 표준

5.2.1 부호화 크기



$$N = (-1)^{a_{n-1}} \times \sum_{i=0}^{n-2} a_i \times 2^i$$

부호 비트(sign bit): $S = a_{n-1}$

[예제 5-1] 19과 -19에 대한 8비트 부호화 크기는?

- +19: **0**_001_0011: 0001_0011 $(-1)^0 \times (2^4 + 2^1 + 2^0)$
- -19: **1**_001_0011: 1001_0011 $(-1)^1 \times (2^4 + 2^1 + 2^0)$

n비트 표현 범위: $-(2^{n-1}-1) \sim +(2^{n-1}-1)$

특징

- 0이 두 개: 0000_0000, 1000_0000
- 덧셈, 뺄셈할 때 부호를 별도로 고려해야 한다.

5.2.2 보수 오직 1, 2의 보수만 다룸

✕ R 진법의 수 N에 대한 보수(complement)

- (R-1)의 보수: $N + C_{R-1} = R^n - 1$
- R의 보수: $N + C_R = R^n = 10...0$
- $C_R = C_{R-1} + 1$
- $R^n = 10...0$ (0이 n개)

✕ [예제 5-2] (R-1)의 보수와 R의 보수 구하기

- 3자리 10진수 457
 - 9의 보수: $(1000-1) - 457 = 542$
 - 10의 보수: 9의 보수 + 1 = 543
- 8비트 2진수 0011_1000
 - 1의 보수: $(1_0000_0000 - 1) - 0011_1000 = 1100_0111$
 - 2의 보수: 1의 보수 + 1 = 1100_1000

2진수의 보수

✱ $\left\{ \begin{array}{l} 1\text{의 보수: } 0 \leftrightarrow 1 \text{ (NOT gate)} \\ 2\text{의 보수: } 1\text{의 보수} + 1 \end{array} \right.$

$$\begin{array}{r} 1111_1111 \\ - 0011_1000 \\ \hline 1100_0111 \end{array}$$

[예제 5-3] 8비트 2진수의 보수

1) 0101_1010	1의 보수: 1010_0101	
	2의 보수: 1010_0110	
2) 0000_0000	1의 보수: 1111_1111	0의 표현이 2 개
	2의 보수: 0000_0000	0의 표현이 1 개

$$\begin{array}{r} 11111111 \\ + 00000001 \\ \hline \text{제거} \leftarrow \textcircled{1}00000000 \end{array} \quad (\text{오버플로우랑 유사})$$

음수의 2의 보수: |음수|의 2의 보수 구하기

2의 보수 구하기: ① Signed magnitude 구하기

② 1의 보수

③ 2의 보수

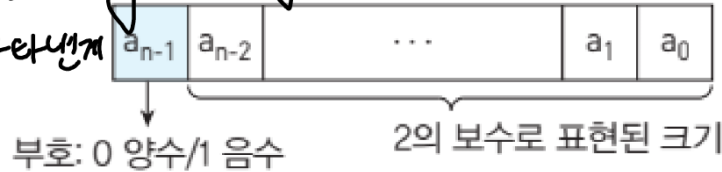
19₁₀: 0001_0011

-19₁₀: 1001_0011

5.2.3 2의 보수 ~~이거 시험나옴~~

양수의 2의 보수 = Signed magnitude

양수는 2진수로 나타낼게
2의 보수임



$$N = -2^{n-1} \times a_{n-1} + \sum_{i=0}^{n-2} 2^i \times a_i$$

[예제 5-4] 10진수 +43과 -43에 대한 8비트 2의 보수 표현

• $43_{10} = 101011_2$

• $+43: 0010_1011 = (-2)^7 \times 0 + (2^5 + 2^3 + 2^1 + 2^0) = (32 + 8 + 2 + 1)$

• $-43: (1) 1_0000_0000 - 0010_1011 = 1101_0101$

(2) $(+43 \text{에 대한 } 1\text{의 보수}) + 1 = 1101_0101$

$(-2)^7 \times 1 + (2^6 + 2^4 + 2^2 + 2^0) = (-128) + (64 + 16 + 4 + 1) = -128 + 85 = -43$

$$\begin{array}{l} 43 / 2 = 21 \dots 1 \\ 21 / 2 = 10 \dots 1 \\ 10 / 2 = 5 \dots 0 \\ 5 / 2 = 2 \dots 1 \\ 2 / 2 = 1 \dots 0 \\ 1 / 2 = 0 \dots 1 \end{array}$$

[예제 5-5] 8비트 정수를 10진수로 변환하기

1) 0110_1010

$64 + 32 + 8 + 2 = 106$

2) 1001_0110

(1) $-128 + (16 + 4 + 2) = -128 + 22 = -106$

(2) 2의 보수: 0110_1010 = 106, $\therefore -106$

부호비트 1: 음수 $\rightarrow$ 그냥 읽기 + - 붙이기

표현 범위

4비트 정수의 값

표 5-2 4비트 정수의 값 ✕

양수	0000	0001(1)	0010(2)	0011(3)	0100(4)	0101(5)	0110(6)	0111(7)	없음
음수		1111(-1)	1110(-2)	1101(-3)	1100(-4)	1011(-5)	1010(-6)	1001(-7)	1000(-8)

n비트 정수의 표현 범위: $-2^{n-1} \sim +(2^{n-1}-1)$

표 5-3 정수 표현 범위 ✕

비트 수	8비트 <i>char</i>	16비트 <i>short</i>	32비트 <i>int</i>
표현 범위	-128 ~ 127	-32,768 ~ 32,767	-2,146,483,648 ~ 2,146,483,647

부호 확장

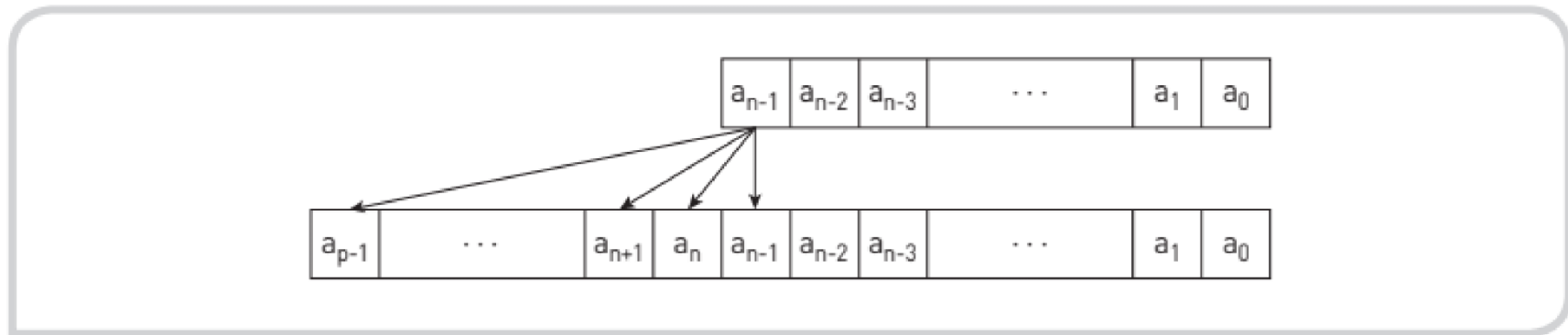


그림 5-4 비트 확장

양수: 앞을 0으로 채움

음수: 앞을 1로 채움

부호 확장 (sign expansion)

- n 비트 정수를 p 비트로 확장 ($n < p$)
- 늘어나는 비트를 부호 비트로 채워야 크기가 변하지 않는다.

[예제 5-6] $+106$ 과 -106 에 대한 16비트 표현

- $+106_{10} = 0100_1010_2 = 0000_0000_0100_1010_2$
- $-106_{10} = 1011_0110_2 = 1111_1111_1011_0110_2$

※ 2의 보수가 표준이 된 이유: 가산기만으로 $+$, $-$ 를 나타낼 수 있기 때문

5.3 논리 연산

논리 연산

- 데이터를 부호 없는 수(unsigned number)로 취급한다.

학습 목표

- 레지스터의 데이터에 대한 논리 연산을 계산할 수 있다.
- 논리 연산의 활용 방법을 이해한다.

내용 (단어 크기를 8bit로 가정)

- 5.3.1 NOT 연산
- 5.3.2 AND 연산
- 5.3.3 OR 연산
- 5.3.4 XOR 연산

5.3.1 NOT 연산 (1의 보수)

Not 연산

- 오퍼랜드의 각 비트를 NOT
- 명령어: NOT R // $R \leftarrow R$ 에 대한 1의 보수

[예제 5-7] R0 = 0010_1011일 때, NOT R0 실행 결과는?

- 명령어 NOT R0
- 실행 결과 R0 = 1101_0100

5.3.2 AND 연산 둘다 1이면 1, 아니면 0

AND 연산

- 비트 단위로 AND 연산을 수행한다.
- 명령어 AND R1, R2 // $R1 \leftarrow R1 \text{ AND } R2$

☆ 마스크(mask) 연산(데이터의 특정 부분만 남기고 나머지를 0으로 하는 연산)

- 특정 비트를 0으로 만든다.

☆ R2의 값 = 마스크 패턴

[예제] AND R1, R2 실행 결과는?

(1) 예제 5-8

R1 = 1010_0110
AND R2 = 1110_0011 ← Mask
R1 = 1010_0010 Pattern

(2) 예제 5-9

R1 = 1010_0110
AND R2 = 0000_1111
R1 = 0000_0110

5.3.3 OR 연산 *0 이 2개이면 0, 아니면 1*

OR 연산

- 비트 단위로 OR 연산을 수행한다.
- 명령어 OR R1, R2 // $R1 \leftarrow R1 \text{ OR } R2$

~~선택적~~ 선택적 세트(selective set) 연산

- 특정 비트를 1로 만든다.

[예제 5-10] OR R1, R2 실행 결과는?

$$\begin{array}{r} R1 = 1010_0110 \\ \text{OR } R2 = 1110_0011 \\ \hline R1 = 1110_0111 \end{array}$$

$k = k + 32;$

$\rightarrow k = k \text{ II } 0x20;$

이전 성능 낮은 시스템에서 속도 더 잘나옴

[예제 5-11] 아스키코드 대문자를 소문자로 바꾸기

$$\begin{array}{r} R1 = 0100_0001 \\ \text{OR } R2 = 0010_0000 \\ \hline R1 = 0110_0001 \end{array} \quad \begin{array}{l} R1 = 'A' \\ \\ R1 = 'a' \end{array}$$

패러디 체크에서 쓰기 5.3.4 XOR 연산

다르면 1, 같으면 0
여러 비트: 1의 개수가 홀수면 1

XOR 연산

- 비트 단위로 XOR 연산을 수행한다.
- 명령어 `XOR R1, R2` // $R1 \leftarrow R1 \text{ XOR } R2$

☆선택적 보수 (selective complement) 연산

- 특정 비트를 보수로 만든다.
- 자신에 대한 XOR 연산 결과는 항상 0000_0000

[예제 5-12] XOR R1, R2 실행 결과는?

$$\begin{array}{r} R1 = 1010_0110 \\ \text{XOR } R2 = 0000_1111 \\ \hline R1 = 1010_1001 \end{array}$$

XOR 연산 예제

[예제 5-13] XOR 연산 연속 실행

- R1 = 1010_0110
- R2 = 0000_1111

- XOR R1, R2

$$\begin{array}{r} \text{R1} = 1010_0110 \\ \text{XOR } \text{R2} = 0000_1111 \\ \hline \text{R1} = 1010_1001 \end{array}$$

- XOR R1, R1

$$\begin{array}{r} \text{R1} = 1010_1001 \\ \text{XOR } \text{R1} = 1010_1001 \\ \hline \text{R1} = 0000_0000 \end{array}$$

❖ 체크섬(checksum)

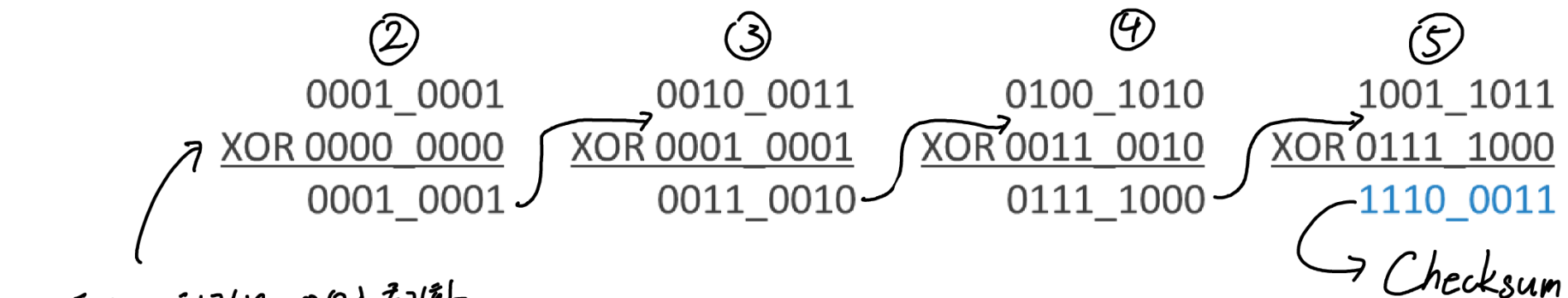
체크섬

→ 일련의 데이터에 대하여 XOR 연산으로 만들어진 값

- 데이터 무결성 확인용

데이터 전체에 대한 패러미터-항상 있음

[11, 23, 4A, 9B]에 대한 체크섬은?



① 첫번째 체크섬은 0으로 초기화

⑥ 송신부는 1110_0011 전송

⑦ 수신부는 받은 데이터로 만든 체크섬이랑

받은 체크섬을 비교: 같으면 데이터 전송이 올바른 것

5.4 시프트 연산

시프트 (shift) 연산

- 왼쪽이나 오른쪽으로 한 비트씩 자리 이동

학습 목표

- 데이터에 대한 시프트 연산을 계산할 수 있다.

내용

- 5.4.1 논리시프트
- 5.4.2 산술시프트
- 5.4.3 회전

5.4.1 논리시프트 : 단순히 0과 1로만 간주 (부호비트 무시)

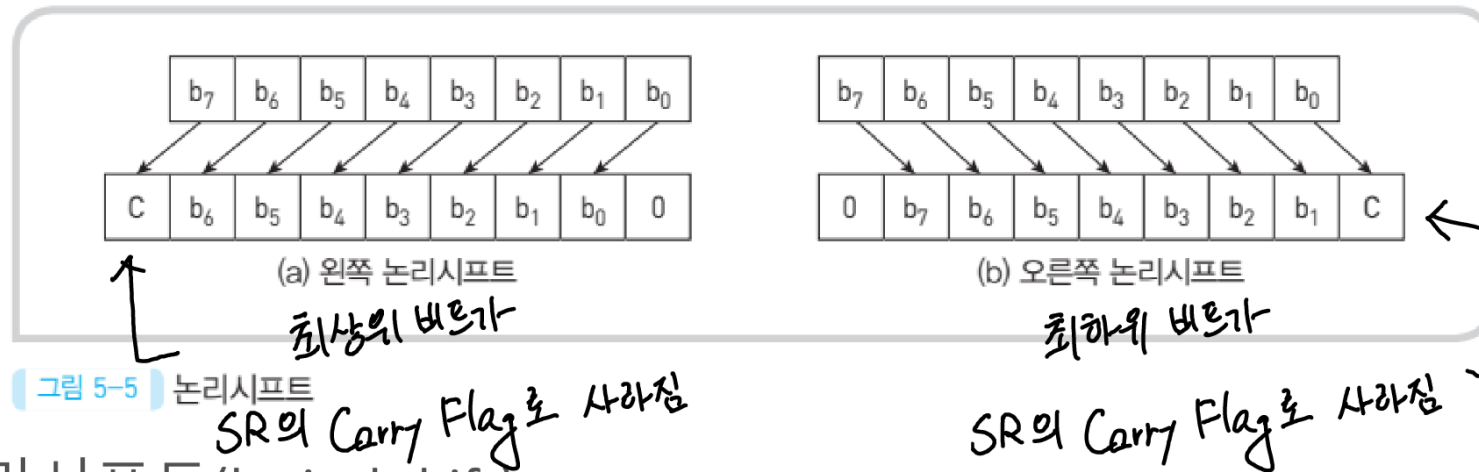


그림 5-5 논리시프트

논리시프트(logical shift)

- 왼쪽 논리시프트: SHL R1 // 한 비트씩 왼쪽으로 자리이동
- 오른쪽 논리시프트: SHR R1 // 한 비트씩 오른쪽으로 자리이동
- ★ 새로 채워지는 비트의 값은 0
- 제거되는 비트는 상태 레지스터의 Carry 플래그에 저장

[예제 5-14] 명령어 실행 결과는?

1) SHL R1 (R1=0010_0110)

C= 0 R1= 0100_1100

2) SHR R2 (R2=1110_0011)

C= 1 R2= 0111_0001

5.4.2 산술시프트 : 2진수에 $\times$ or $\div 2$ 연산, 부호 취급

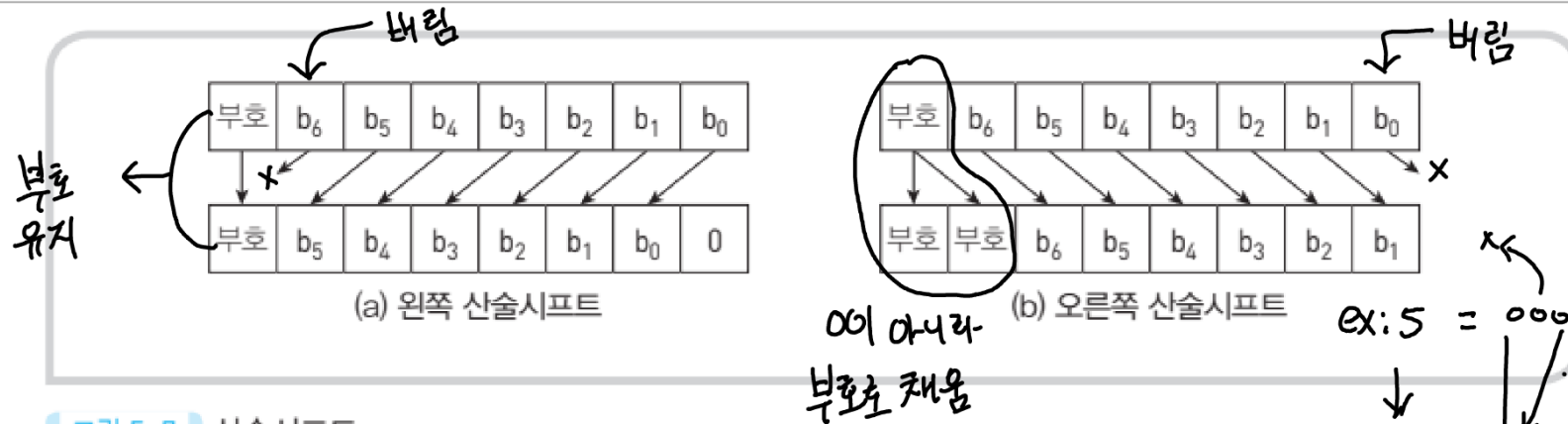


그림 5-7 산술시프트

산술시프트(arithmetic shift)

- 왼쪽 산술시프트: ASL R1 // $R1 \leftarrow R1 \times 2$. 왼쪽에 0을 채움
- 오른쪽 산술시프트: ASR R1 // $R1 \leftarrow R1 \div 2$. 부호 비트를 채움
- 부호 비트는 고정

ex: $5 = 0000 - 0101$

$5 \ll 1 \rightarrow 0000 - 1010 = 10$

$5 \ll 2 : 20 \Rightarrow 5 \times 2^2$

$\therefore Q \ll b$

$= Q \times 2^b$

$Q \gg b$

$= Q \times 2^{-b}$

[예제 5-15] 명령어 실행 결과는? 곱하기2, 나누기2 효과가 있는가?

- 1) ASL R1 ($R1 = 0010_0110_2 = 38_{10}$) $R1 = \underline{\hspace{2cm}}$ (2진수) = $\underline{\hspace{2cm}}$ (10진수)
- 2) ASR R2 ($R2 = 1110_0010_2 = -30_{10}$) $R2 = \underline{\hspace{2cm}}$ (2진수) = $\underline{\hspace{2cm}}$ (10진수)

5.4.3 회전 *Rotation*

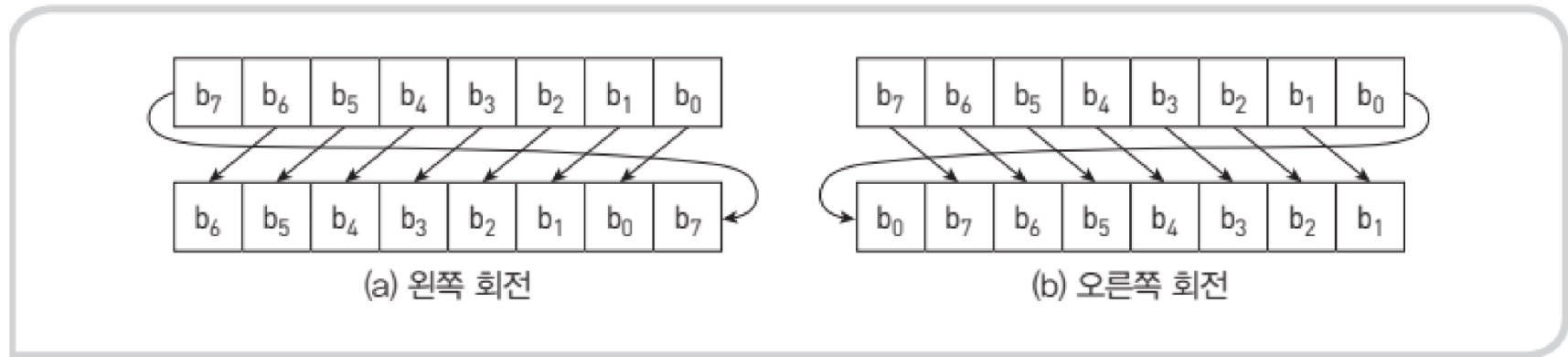


그림 5-9 회전

밀려나가는 비트가 반대 방향의 끝에 채워지는 연산

[예제 5-16] 회전 연산의 결과는?

1) ROL R1 (R1 = 0010_0110)

R1 = _____

2) ROR R2 (R2 = 1110_0011)

R2 = _____

X멀티플렉서 시프트 회로X

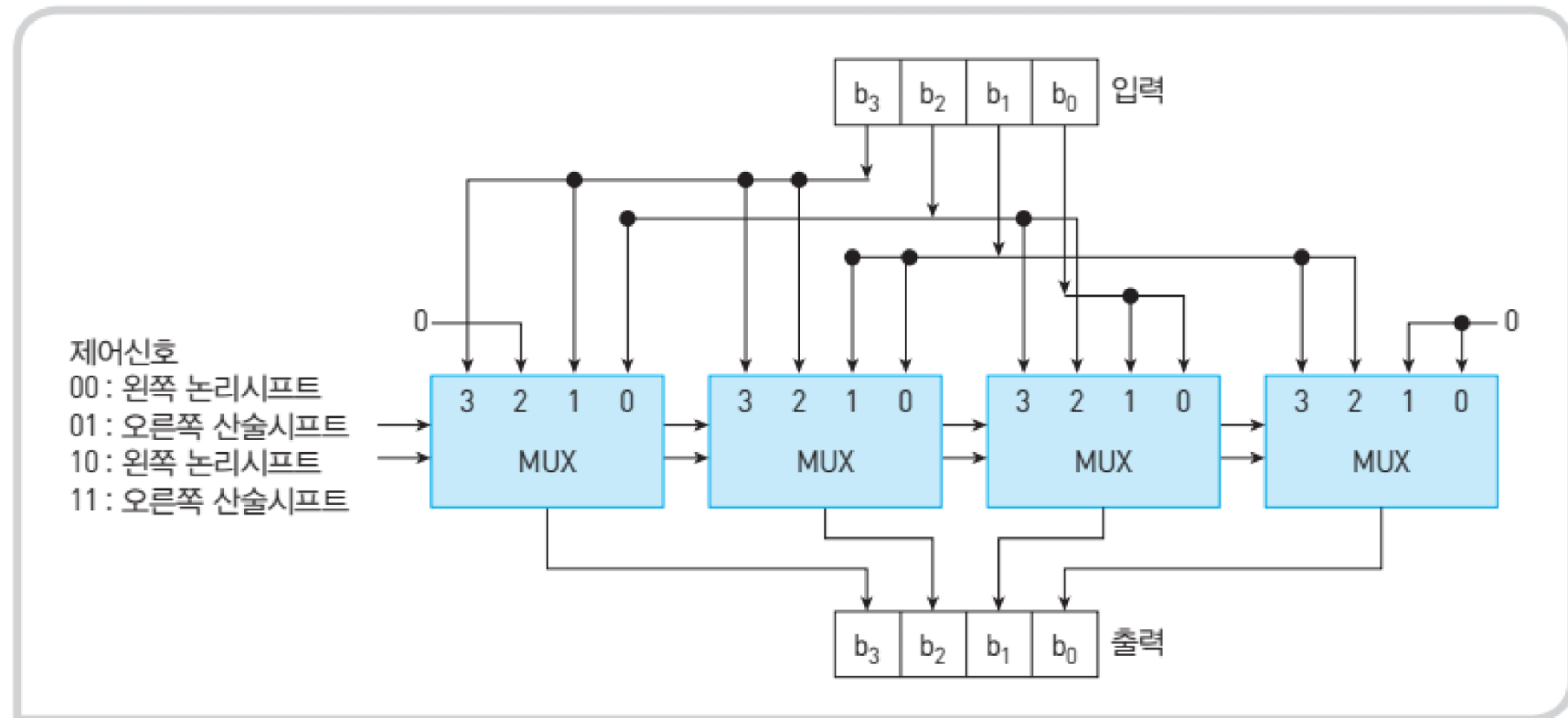


그림 5-11 멀티플렉서를 사용한 시프트 회로

5.5 산술 연산

산술 연산

- 양수, 음수를 구분하지 않는다.
- 2의 보수로 해석한다.

학습 목표

- 2진수에 대한 사칙연산을 할 수 있다.
- 오버플로우가 발생하는 이유를 설명할 수 있다.

내용

- 5.5.1 단항 연산
- 5.5.2 덧셈과 뺄셈
- 5.5.3 곱셈
- 5.5.4 나눗셈

5.5.1 단항 연산

단항 연산

- NEG R *Negate* // $R \leftarrow -R$ (2의 보수) $-R$
- INC R *Increment* // $R \leftarrow R+1$ (증가) $R++$
- DEC R *Decrement* // $R \leftarrow R-1$ (감소) $R--$

[예제 5-17] 명령어 실행 결과는?

- 1) R1=0010_1100, NEG R1 R1 = _____
- 2) R2=0011_1111, INC R2 R2 = _____
- 3) R3=0000_0000, DEC R3 R3 = _____

데이터를 부호 없이 계산후 결과값을 정수·부호있는 수로 보면 됨
(연산 시 부호 신경 안써도 됨)

5.5.2 덧셈과 뺄셈

덧셈: `ADD R1, R2` `// R1 ← R1+R2`

뺄셈: `SUB R1, R2` `// R1 ← R1-R2 = R1 + (R2에 대한 2의 보수)` ★

뺄셈 = 2의 보수 덧셈

[예제 5-18] 덧셈 결과 해석

$$\begin{array}{r} 1000_1101 \\ + 0110_0101 \\ \hline 1111_0010 \end{array}$$

(a) 2진수 덧셈

$$\begin{array}{r} 141 \\ + 101 \\ \hline 242 \end{array}$$

(b) 부호 없는 수 해석

$$\begin{array}{r} -115 \\ + 101 \\ \hline -14 \end{array}$$

(c) 정수 해석

[예제 5-19] 뺄셈

$$\begin{array}{r} 0100_1110 \\ - 0001_1000 \\ \hline \end{array}$$

(a) 뺄셈

자리올림: SR의 Carry Flag로 저장

$$\begin{array}{r} 0100_1110 \\ + 1110_1000 \\ \hline C \leftarrow 10011_0110 \end{array}$$

(b) 2의 보수 덧셈

$$\begin{array}{r} 78 \\ - 24 \\ \hline 54 \end{array}$$

(c) 해석

오버플로우: 연산 결과가 표시 범위 벗어날 때 발생

오버플로우(Overflow)

- 연산 결과가 수의 표현 범위를 초과하는 현상
- $\text{Overflow} = C_n \oplus C_{n-1} \Rightarrow$ 자리올림 비트의 최상위 2개가 서로 다르면 오버플로우
 $\uparrow$ XOR

(8비트)
[예제 5-20] 오버플로우가 발생하는지 예측하라. 플래그 값은?

(1) $89+45$

(2) $89-45$

(3) $-89+45$

(4) $-89-45$

C_8C_7	C_8C_7	C_8C_7	C_8C_7
01111 001	11010 011	00101 001	10000 111 ← 자리올림
0101_1001	0101_1001	1010_0111	1010_0111
+ 0010_1101	+ 1100_0011	+ 0010_1101	+ 1101_0011
1000_0110 = -122	0010_1100 = 44	1101_0100 = -44	0111_1010 = 122
(a) $89+45$	(b) $89-45$	(c) $-89+45$	(d) $-89-45$

그림 5-14 정수 덧셈과 오버플로우

상태 플래그

[예제 5-21] 89+45 덧셈 후 상태 플래그의 값은?

$$89+45=132>127$$

C_8C_7

01111 001

0101_1001

+0010 1101

1000_0110

Sign flag= 1

Zero flag= 0

Carry flag= 0 C는 최종 비트

Parity flag= 1 (even 일때)

Overflow flag= 1

병렬 가산기

Add/Sub = 0일 때, 덧셈: $C_4 S_3 S_2 S_1 S_0 = A + B$

Add/Sub = 1일 때, 뺄셈: $C_4 S_3 S_2 S_1 S_0 = A + \neg B + 1 = A - B$

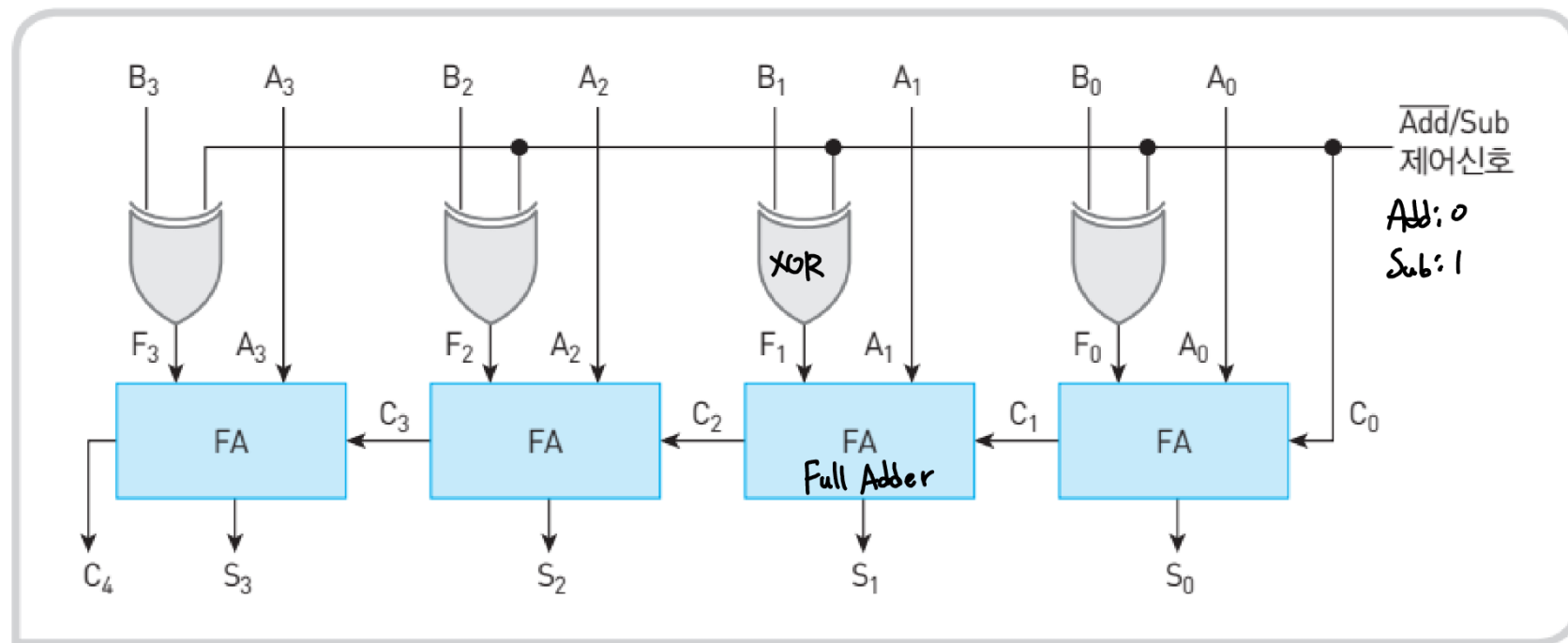


그림 5-15 병렬 덧셈/뺄셈기 → 덧셈기로 가감산 가능

X 5.5.3 곱셈 X

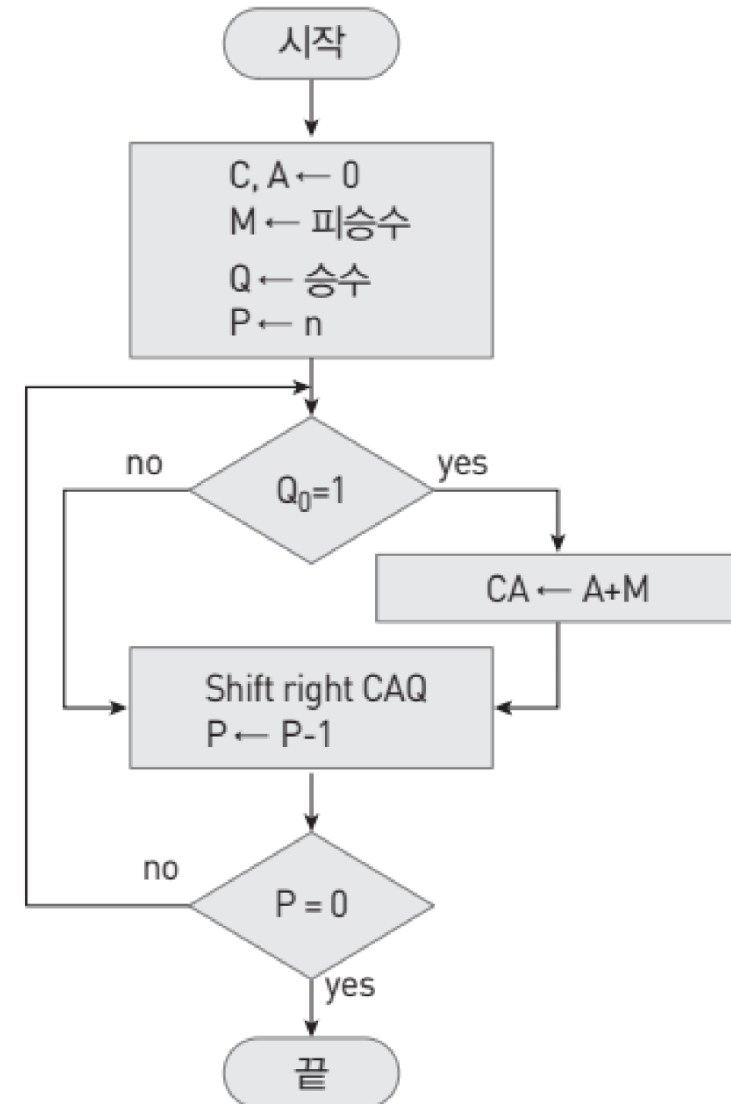
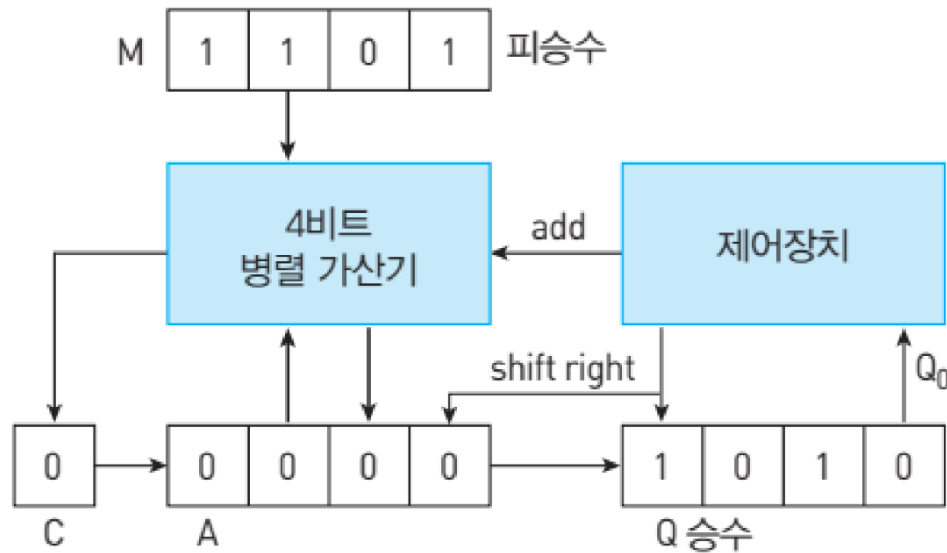
n비트 곱셈

- $a \times b = \text{곱}$ (product, $2n$ 비트)
- a: 피승수(multiplicand), b: 승수(multiplier)

부호 없는 수 곱셈

1101	피승수 (13)
$\times 1010$	승수 (10)
1101 $\times 0 \rightarrow$ 0000	
1101 $\times 1 \rightarrow$ 1101	부분 곱
1101 $\times 0 \rightarrow$ 0000	
1101 $\times 1 \rightarrow$ +1101	
10000010	곱 (130)

X 곱셈 알고리즘 X



✕ 곱셈 알고리즘 ✕

[예제 5-22] 13 x 10

	M	C	A	Q	P _←	
[초기 상태]	1101	0	0000	1010	4	// 초기 상태 _← // Q ₀ = 0 _←
[오른쪽 시프트]	1101	0	0000	0101	3	// Shift CAQ, P ← P - 1 // Q ₀ = 1 _←
[부분 곱 계산]	1101	0	1101	0101	3	// CA ← A + M _←
[오른쪽 시프트]	1101	0	0110	1010	2	// Shift CAQ, P ← P - 1 // Q ₀ = 0 _←
[오른쪽 시프트]	1101	0	0011	0101	1	// Shift CAQ, P ← P - 1 // Q ₀ = 1 _←
[부분 곱 계산]	1101	1	0000	0101	1	// CA ← A + M _←
[오른쪽 시프트]	1101	0	1000	0010	0	// Shift CAQ, P ← P - 1

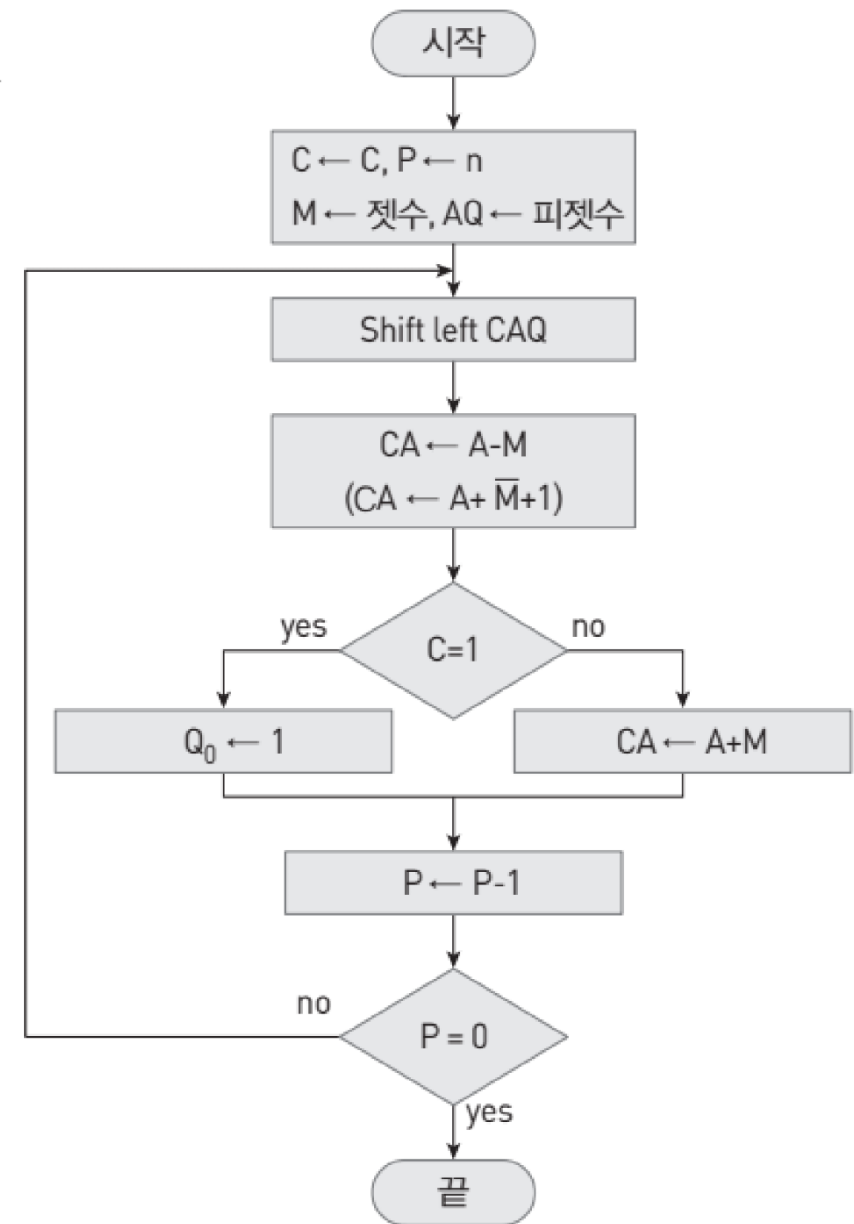
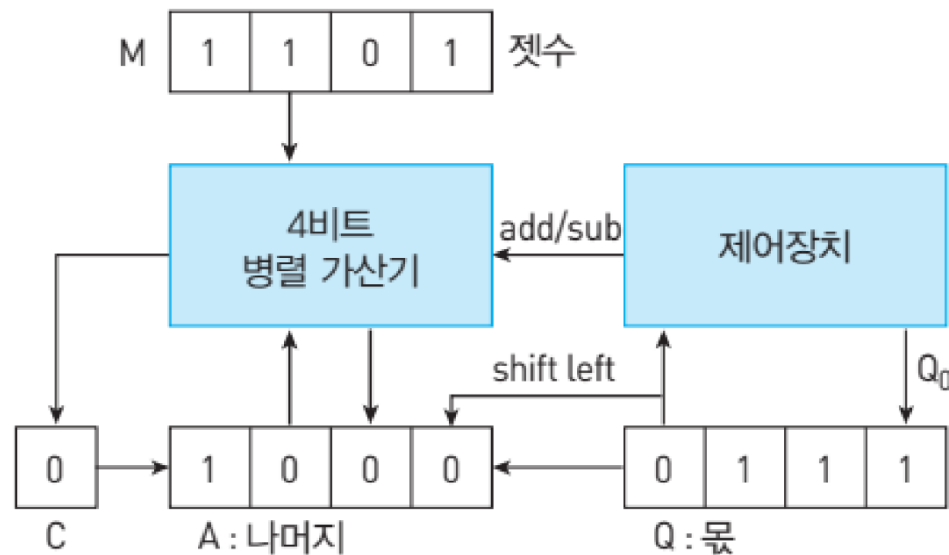
5.5.4 나눗셈

$2n$ 비트 / n 비트 나눗셈 = n 비트 몫 ... n 비트 나머지

- a/b = 몫 (quotient) ... 나머지 (remainder)
- a : 피젯수 (dividend), b : 젯수 (divisor)

		1010	몫 (10)
젯수 (13)	1101	) 10000111	피젯수 (135)
		- 1101	
부분 나머지		0011 1	
		- 0000	
		0111 1	
		- 1101	
		0010 1	
		- 0000	
		0101	나머지 (5)

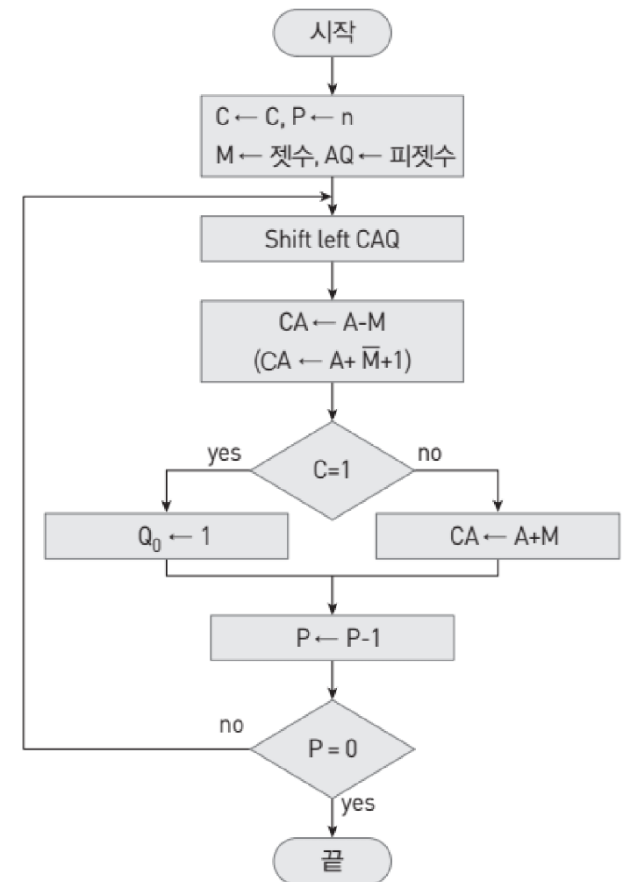
나눗셈 알고리즘



부호 없는 수 나눗셈 알고리즘

[예제 5-23] 135/13

	M	C	A	Q	P	
[초기 상태]	1101	0	1000	0111	4	// 초기 상태
[왼쪽 시프트]	1101	1	0000	1110	4	// Shift left CAQ
[A-M]	1101	1	0011	1110	4	// $A \leftarrow A + \bar{M} + 1$, $C=1$ ($A \geq M$)
[$Q_0 \leftarrow 1$]	1101	1	0011	1111	3	// $Q_0 \leftarrow 1$, $P \leftarrow P-1$
[왼쪽 시프트]	1101	0	0111	1110	3	// Shift left CAQ
[A-M]	1101	0	1010	1110	3	// $A \leftarrow A + \bar{M} + 1$, $C=0$ ($A < M$)
[A+M, $Q_0 \leftarrow 0$]	1101	1	0111	1110	2	// $Q_0 \leftarrow 0$, $CA \leftarrow A+M$, $P \leftarrow P-1$
[왼쪽 시프트]	1101	0	1111	1100	2	// Shift left CAQ
[A-M]	1101	1	0010	1100	2	// $A \leftarrow A + \bar{M} + 1$, $C=1$ ($A \geq M$)
[$Q_0 \leftarrow 1$]	1101	1	0010	1101	1	// $Q_0 \leftarrow 1$, $P \leftarrow P-1$
[왼쪽 시프트]	1101	0	0101	1010	1	// Shift left CAQ
[A-M]	1101	0	1000	1010	1	// $A \leftarrow A + \bar{M} + 1$, $C=0$ ($A < M$)
[A+M, $Q_0 \leftarrow 0$]	1101	1	0101	1010	0	// $Q_0 \leftarrow 0$, $CA \leftarrow A+M$, $P \leftarrow P-1$



5.6 실수

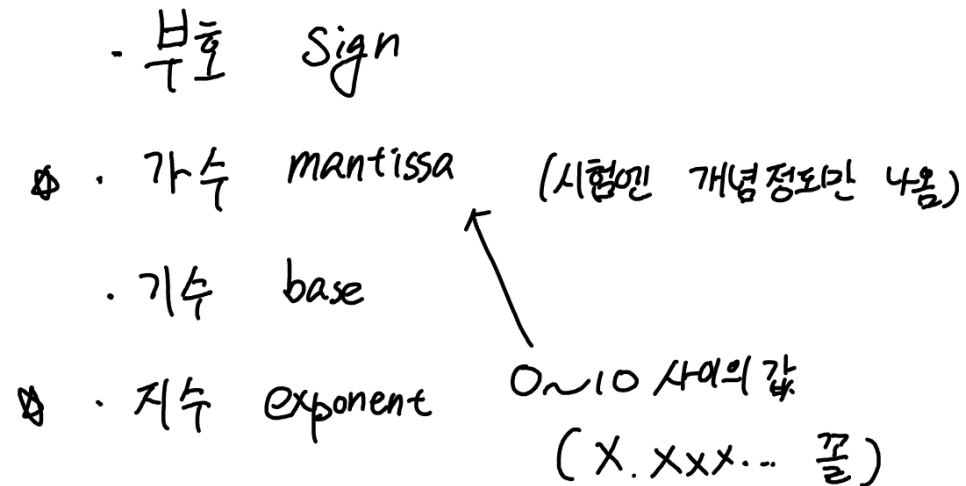
학습 목표

- 실수(floating-point number)를 표현하는 원리를 이해한다.
- 표준 규격에 따라 실수 값을 해석할 수 있다.

내용

- 5.6.1 부동소수점 표현
- 5.6.2 IEEE754 형식

· 부호 sign
· 가수 mantissa (시험엔 개념정도만 나옴)
· 기수 base
· 지수 exponent (0~10 사이의 값)
(X.XXX... 꼴)



↓ 플 부 (float)

5.6.1 부동 소수점 표현

과학 표기(scientific notation)

- $\pm \text{significant} \times \text{Base}^{\text{exponent}}$ (부호·가수 $\times$ 기수^{지수})
- significant = mantissa = fraction

10진수 과학 표기 예

- $976,000,000,000 = 9.76 \times 10^{11}$
- $-0.000000000000976 = -9.76 \times 10^{-12}$

↙ 유효숫자 부분

부호: + 가수: 9.76 지수: 11
부호: - 가수: 9.76 지수: -12
base: 10 mantissa exponent

2진수 예

- $0.1101 \times 2^2 = 11.010 \times 2^0 = 110.10 \times 2^{-1}$
- 정규화(normalize) 필요

× 가수 정규화(normalized mantissa) ×

가수 정규화

- $(1.bbb...b \times 2^{\pm E})$ 형식이 되도록 지수 조정
- 1.은 표현에서 생략 (항상 1.0은 있는 것으로 취급)
- 가수를 표현하는 비트 영역 최대 활용
- 숫자 0은 모든 비트를 0으로 채움

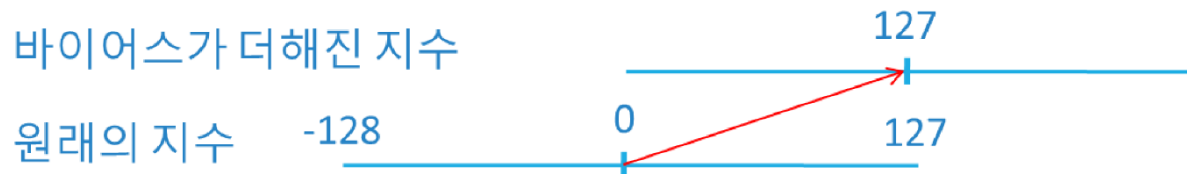
[예제 5-24] 2진수 101.1010_1111에 대한 가수 표현은?
(단, 가수를 8비트로 표현)

- $101.1010_1111 = 1.0110_1011_11 \times 2^{+2}$
- 1.은 생략
- 가수: 0110_1011만 표현
- 하위 두 비트는 없어짐

바이어스 지수(biased exponent)

바이어스 지수

- 표현 = 지수 + bias // 지수를 양수로 변환
- 실제 지수 = 표현 - bias
- 예) 지수 영역이 8비트일 때, $\text{bias} = 127_{10} = 0111_1111$



- 이유? 덧셈 뺄셈을 할 때, 양수 영역에서 지수를 비교하기 위하여

[예제 5-25] 101.101011에 대한 지수 표현 값은?
(단, 지수에 8비트 할당, $\text{bias} = 0111_1111$)

- 가수 정규화: $101.1010_11 = 1.0110_1011 \times 2^2$
- 지수 값 = $2_{10} = 0000_0010$
- 지수 표현 = $0000_0010 + 0111_1111 = 1000_0001$.

5.6.2 IEEE754 형식

I Triple E 754



(a) 단정도 형식 = 32bit = Single precision format



(b) 배정도 형식 = 64bit = double precision format

그림 5-23 IEEE754 표준 형식

표 5-4 IEEE754 표준 형식의 해석

단정도 형식			배정도 형식		
지수	가수	해석	지수	가수	해석
255	$\neq 0$	NaN	2047	$\neq 0$	NaN
255	0	$(-1)^S \infty$	2047	0	$(-1)^S \infty$
$0 < e < 255$	-	$(-1)^S 2^{e-127} (1.f)$	$0 < e < 2047$	-	$(-1)^S 2^{e-1023} (1.f)$
0	$\neq 0$	$(-1)^S 2^{e-126} (0.f)$	0	$\neq 0$	$(-1)^S 2^{e-1022} (0.f)$
0	0	$(-1)^S 0$	0	0	$(-1)^S 0$

× 단정도 실수 ×

[예제 5-26]

- 가장 정밀한 양수
- 비트 패턴: 부호 = 0, 지수 = 0, 가수 = 000_0000_0000_0000_0000_0001
- 크기:
$$\begin{aligned} & (-1)^s 2^{e-126}(0.f) \\ &= (-1)^0 2^{0-126}(0.00000000000000000000000001) \\ &= 2^{-126-23} = 1 \times 2^{-149} \end{aligned}$$
- 가장 큰 양수
- 비트 패턴: 부호 = 0, 지수 = 254, 가수 = 111_1111_1111_1111_1111_1111
- 크기:
$$\begin{aligned} & (-1)^s 2^{e-127}(1.f) \\ &= (-1)^0 2^{254-127}(1.111_1111_1111_1111_1111_1111) \\ &= 2^{127}(0.1111_1111_1111_1111_1111_1111) \times 2 \\ &= (1 - 2^{-24}) \times 2^{128} \end{aligned}$$

이 뒤로는 어려워서 안함

IEEE754 예

[예제 5-27] 다음 단정도 실수의 10진수 값은?

1100_0001_0101_0000_0000_0000_0000

- 비트 패턴: (부호)1_(지수)10000010_(가수)101000000000000000000000
- 부호: 1
- 지수: $e = 1000_0010 = 130_{10}$
- 가수: $M = 1.101$
- $N = (-1)^1 \times 1.101 \times 2^{130-127} = -1.101 \times 2^3 = -1101 = -13_{10}$

5.7 실수 연산

실수 연산의 예외

- 지수 오버플로우(exponent overflow)
 - 양의 지수값이 최대 지수값을 초과한 경우
 - 연산 결과는 $+\infty$ 또는 $-\infty$ 로 표시된다.
- 지수 언더플로우(exponent underflow)
 - 음의 지수값이 최소 지수값보다 적은 경우
 - 연산 결과는 0으로 표시된다.
- 가수 오버플로우(significant overflow)
 - 최상위 비트에서 자리올림수가 발생한 경우
 - 재정렬(realignment): 가수를 왼쪽으로 쉬프트하고 지수를 조정
- 가수 언더플로우(significant underflow)
 - 가수를 조정할 때 가수가 오른쪽으로 밀려 잘려나가는 경우
 - 연산 결과의 최하위 비트를 반올림한다.

5.7.1 덧셈과 뺄셈

$$\begin{array}{r} x = 0.011111 \times 2^1 \\ y = 1.1111 \times 2^1 \\ \hline x+y = 10.0110 \times 2^1 = 100.011 \end{array}$$

(a) 작은 수를 큰 수에 맞춤

$$\begin{array}{r} x = 1.1111 \times 2^{-1} \\ y = 11.1100 \times 2^{-1} \\ \hline x+y = 10.11011 \times 2^{-1} = 1.0110 \end{array}$$

(b) 큰 수를 작은 수에 맞춤

그림 5-24 실수 덧셈 예

$$(x = S_x \times 2^{E_x}) \pm (y = S_y \times 2^{E_y})$$

- 제로 검사
 - x 와 y 가 0인지 검사한다
 - 만일 둘 중 하나가 0이면, 덧셈과 뺄셈을 수행할 필요가 없다.
- 정렬
 - 지수를 비교하여 작은 수의 지수를 큰 수의 지수와 같아지도록 조정
 - 작은 수의 지수를 증가시키고, 가수를 오른쪽으로 시프트 한다.
- 가수 덧셈/뺄셈” 가수를 서로 더하거나 뺀다.
- 정규화: 결과를 정규화 한다.

5.7.2 곱셈과 나눗셈

$$x = S_x \times 2^{E_x}, y = S_y \times 2^{E_y}$$
$$x \times y = (S_x \times S_y) \times 2^{E_x + E_y} \quad x / y = (S_x / S_y) \times 2^{E_x - E_y}$$

곱셈

제로 검사

- 만일 x 와 y 둘 중 하나가 0이면, 곱은 0이다.

지수 조정

- 지수 E_x 와 E_y 를 더하고 바이어스를 뺀다.

가수 곱셈

- 부호를 고려하여 s_x 와 s_y 를 곱한다.

정규화

- 곱셈 결과를 정규화 한다.

나눗셈

제로 검사

- 만일 x 가 0이면, 결과는 0이고, y 가 0이면 결과는 NaN으로 설정하고 계산을 종료한다.

지수 조정

- 지수 E_x 에서 E_y 를 빼고 바이어스를 더한다.

가수 나눗셈

- 부호를 고려하여 s_x 에서 s_y 를 나눈다.

정규화

- 나눗셈 결과를 정규화 한다.

5.8 요약

5.2 정수 표현

- 정수 표현 방법: 부호화 크기, 2의 보수(표준)

5.3 논리 연산

- NOT 연산, AND 연산 (mask), OR 연산 (selective set), XOR 연산 (selective complement)

5.4 시프트 연산

- 논리시프트, 산술시프트, 회전

5.5 정수 산술 연산

- 단항 연산: 음수 만들기, 증가, 감소
- 덧셈과 뺄셈: 2의 보수에 의한 연산, 오버플로우
- 곱셈과 나눗셈: 부호 없는 수

5.6 실수

- 부동소수점 표현: 정규화 가수, 바이어스 지수
- IEEE 754 형식

5.7 실수 연산

- 덧셈과 뺄셈: 지수 맞추기, 재정렬
- 곱셈과 나눗셈: 바이어스 조정